Research Workflow Agent 项目模拟面试题

按当前技术版本校准 + 20 个高频问题与标准回答 + 面经问法风格参考

定位	面向科研选题调研、论文精读与综述写作的闭环式 Research Workflow Agent。
技术栈	Python / LangGraph / FastAPI / RAG / Multi-Agent / MCP / Skills
使用方式	优先按“当前已落地功能 + 已完成设计但仍在灰度实现”的口径作答，避免把未来 phase 全部说成已经生产可用。

结论先说：这条项目线作为 Agent / RAG / Workflow 项目是成立的，但面试里一定要把“已实现”“正在实现”“后续 Phase 设计”分开讲。就当前版本，更稳的说法是：StateGraph 工作流、Workspace/Artifact、结构化抽取与 reviewer 思路是主干；Multi-Agent / MCP / Skills 是升级层，先按兼容式灰度接入，不要一上来讲成完整生产级平台。

一、当前功能是否符合预期：面试时的真实口径

整体判断：方向是对的，项目题材也符合当前牛客/面经里常见的 Agent 开发问法；但从答题策略上，应该把项目讲成“以 LangGraph 为控制平面的闭环研究工作流，先完成工作流底座与结构化中间产物，再逐步补强 RAG、Reviewer、Trace、Multi-Agent 与 MCP 扩展层”。

不要把所有未来规划都说成已经 fully landed。对面试官来说，最在意的是：你是否清楚系统边界、是否知道为什么要这样拆、每一层解决了什么问题、当前版本的真实完成度是什么。

模块	建议口径	面试风险
LangGraph 工作流	可以说已实现主干，负责状态流转、阶段编排与失败/重试控制。	低
Workspace / Artifact	可以说已实现或至少已明确落到接口与 schema，支撑中间产物沉淀与复用。	低
RAG / 检索	更稳的说法是已做基础版，正在增强为分层检索 + 多阶段重排 + 结构化 RagResult。	中
Reviewer / Trace / Eval	如果已有节点与 schema，可说已	中

	进入闭环质量控制阶段；没有就说已设计并部分接入。	
Multi-Agent / MCP / Skills	建议说已完成兼容式架构设计，正在以 hybrid 模式灰度接入。	高

二、20 个高频模拟面试题（按当前技术版本作答）

1. 你先用一句话介绍这个项目。

标准解答：这是一个面向科研调研与综述写作场景的闭环式 Research Workflow Agent。它不是单纯做问答，而是把需求澄清、检索规划、论文获取、结构化抽取、跨文献整合、审查修订和报告生成串成一条可追踪、可恢复、可复用的工作流。

2. 这个项目和普通的 RAG 问答系统有什么本质区别?

标准解答：普通 RAG 更像“检索后回答”，目标通常是单轮或短流程问答；这个项目的目标是“研究任务完成”。所以我把系统设计成显式阶段工作流，产出不只是答案，而是 research brief、query plan、paper cards、comparison matrix、review log 和 report draft 这些中间产物。也就是说，它更接近 research workflow orchestration，而不是一个聊天机器人。

3. 为什么你选 LangGraph，而不是只用 LangChain?

标准解答：因为这个项目天然是有状态、长流程、可中断恢复的。LangChain 更适合快速串工具链和 prompt；而 LangGraph 更适合显式表示节点、边、状态、失败回退、条件分支和人工确认节点。这个项目里，StateGraph 更像控制平面：它把外层工作流固定下来，避免系统一开始就过度 agent 化。

4. 你为什么把流程拆成 Scope → Search → Read → Synthesize → Review → Write?

标准解答：这是按研究任务的真实生命周期来拆的。Scope 解决目标不清；Search 解决去哪找、找什么；Read 解决单篇信息抽取；Synthesize 解决跨文献整合；Review 解决覆盖不足、证据薄弱和引用错误；Write 才是最后把中间产物组织成报告。这样拆的好处是每一步都有清晰输入输出，也便于后面做 trace 和 eval。

5. 当前版本到底实现到了哪一步?

标准解答：我会诚实分层说。当前主干是：基于 LangGraph 的研究任务工作流、ResearchTask / Workspace / Artifact 这套中间产物机制、以及结构化 paper card / reviewer 闭环思路。RAG 部分不是停留在字符串检索，而是在往结构化 RagResult、分层检索和多阶段重排演进。Multi-Agent、MCP、Skills 是兼容式升级层，当前更适合表述为“已设计并在灰度接入”，而不是全部 production ready。

6. 你为什么要单独设计 ResearchTask、Workspace、Artifact?

标准解答：因为长流程 Agent 最大的问题不是生成一次答案，而是任务能不能持续迭代。ResearchTask 负责任务生命周期，Workspace 负责长期工作区，Artifact 负责把中间结果实体化。这样系统就能把 brief、plan、paper cards、matrix、review feedback、draft 等沉淀下来，而不是把所有东西都塞在上下文窗口里。

7. SearchPlan 这一步你具体做了什么?

标准解答：SearchPlan 不是简单把用户 query 改写一下，而是把模糊需求转成结构化的 ResearchBrief、SubQuestion 和 SearchQuery。它的意义在于：后续检索不是盲搜，而是带着 inclusion / exclusion criteria、输出格式要求和子问题拆解去搜。这样检索结果和最终报告结构之间才有一致性。

8. 你的 RAG 是怎么设计的?

标准解答：按当前版本的目标，我更倾向于 research-oriented RAG，而不是 FAQ RAG。核心路线是 hierarchical + multi-stage：先做 paper-level retrieval，得到候选论文；再在候选论文内部做 chunk-level evidence retrieval；同时在每一层都保留 recall、dedup、rerank 这样的多阶段质量控制。最终返回的不是一段拼接文本，而是结构化的 RagResult。

9. 为什么要做 hierarchical retrieval?

标准解答：因为研究任务里“找论文”和“找证据”是两件事。先找相关论文，再在论文内部找能支撑某个子问题的证据段，这样后面的 PaperCard 抽取和 reviewer 审查才有抓手。如果一开始就只做 chunk 级检索，容易命中零散片段，反而不利于建立论文级候选集合。

10. 为什么还要加 multistage retrieval?

标准解答：因为只分层还不够，研究检索更怕噪声和重复。Multi-stage 的目标是：先高召回，再做 canonical dedup，再做 rerank，最后再细化到 evidence retrieval。这样可以减少同一论文多来源重复、降低高分噪声，并让真正重要的候选论文排到前面。

11. 你为什么强调 PaperCard、ComparisonMatrix 这些结构化中间产物?

标准解答：因为如果直接让模型把几篇论文摘要拼成综述，结果很难审，也很难复用。PaperCard 把单篇论文的任务、方法、数据集、指标、贡献、局限固定下来；ComparisonMatrix 则把跨文献可对比的信息拉平。这样 reviewer 才能检查 coverage gap、unsupported claim 和 citation reachability，而不是对着一段自由文本拍脑袋。

12. 这个项目怎么降低幻觉和错误引用?

标准解答：我不会说单靠 prompt 解决。这个项目里，降低幻觉主要靠四层：第一，检索层提升 paper recall 和 evidence precision；第二，抽取层把引用绑到具体 evidence span；第三，reviewer 检查 unsupported claims 和 citation reachability；第四，必要时触发 re-search 或 rewrite。也就是说，幻觉控制被拆到 retrieval、extraction、review 三层，而不是只靠最后生成阶段。

13. Reviewer 在这个系统里具体做什么?

标准解答：Reviewer 不是润色器，而是质量闸门。它主要审四类问题：coverage gap，也就是有没有重要主题或论文漏掉；unsupported claim，也就是结论有没有证据支撑；citation reachability，也就是引用能不能回指到真实 source；以及 duplication / consistency，也就是结构和结论之间是否重复或冲突。它的输出是 ReviewFeedback 和 RevisionAction，而不是一段模糊评价。

14. 你为什么要做 node wrapper 和 tool trace?

标准解答：因为 Agent 系统最怕“结果出来了，但没人知道它怎么来的”。Node wrapper 会记录每个节点的输入输出、状态、耗时和异常；tool trace 会记录具体调用了什么工具、用了哪些参数、拿回了什么结果。这样一方面便于 debug 和回归测试，另一方面 reviewer、workspace artifact 面板和 internal eval 都能直接复用这些 trace。

15. 你这个项目怎么做 eval?

标准解答：我把 eval 分成三层。第一层是 retrieval eval，看 Recall@K、MRR、nDCG、evidence hit rate；第二层是 reviewer eval，看 unsupported claim、coverage gap、citation 问题的检出率；第三层是 workflow eval，看节点成功率、artifact 完整率、review pass rate 等。也就是说，它不是只评“最终回答像不像”，而是评整个 research workflow 的质量链路。

16. 为什么这个项目后面还要做 multi-agent? 单 agent 不够吗?

标准解答：不是说单 agent 一定不行，而是职责分工清晰后更容易维护和扩展。Planner 负责澄清和规划，Retriever 负责检索，Analyst 负责抽取和整合，Reviewer 负责质量控制。这样做的前提是前面已经有统一的 state、artifact、trace 和 eval；否则多 agent 很容易变成几套 prompt 互相乱传文本，工程价值并不高。

17. 你这个多 Agent 怎么和 Phase 1 兼容?

标准解答：我不会推翻外层 StateGraph。Phase 1 的价值是 control plane，也就是任务状态、阶段编排、artifact 持久化和 SSE/trace 这些底座。Phase 4 的多 Agent 是 collaboration plane：它更多发生在节点内

部或节点上层，由 supervisor 决定节点走 legacy、hybrid 还是 v2 backend。这样可以先双轨并存，再基于 trace 和 eval 决定何时删旧逻辑。

18. MCP 在你的项目里是干什么的?

标准解答：MCP 在这里不是单独的 agent，而是统一外部能力的接入协议层。我会把它放在 mcp_adapter 里，用来管理外部 tools、resources、prompts 的发现、连接、调用和权限控制。这样 agent 不需要直接依赖某个外部工具实现，只需要调用统一 Tool Runtime，底层再决定是本地函数、Corpus Service 还是 MCP server。

19. Skills 和 tool 的区别是什么?

标准解答：Tool 更像单一原子能力，比如检索、读文件、取 PDF；skill 则是高频动作的复用单元，通常是 prompt、工具链和局部工作流的组合。比如 literature scan、paper plan builder、creative reframe 这类更适合做 skill，而不是裸 tool。这样前端既可以自动调用，也可以显式点选运行。

20. 如果你继续做这个项目，下一步最关键的演进点是什么?

标准解答：我会优先做三件事。第一，把兼容式 Phase 4 跑通，也就是 Multi-Agent、MCP、Skills 先双轨接进现有工作流，而不是大改架构；第二，用 internal eval 比较 legacy / hybrid / v2 三套 backend 的质量差异，作为删旧依据；第三，把前端的 Agent Switcher、Skill Palette 和 MCP Catalog 做出来，让系统既支持自动编排，也支持显式调试和半手动运行。

三、面试表达建议（避免翻车）

1. 不要把“Phase 4 架构设计完成”讲成“多 Agent + MCP + Skills 全部生产落地”。面试官追问到接口、trace、回滚、失败路径时很容易穿帮。

2. 讲项目时优先用“控制平面 / 协作平面 / 执行平面”这类工程化语言，少用空泛的“智能体自己会思考”。

3. 所有回答都尽量落到对象和产物：ResearchTask、SearchPlan、RagResult、PaperCard、ReviewFeedback、Artifact、Trace、Eval。

4. 如果被问“你个人做了什么”，不要只说‘我设计了架构’。要补上：阶段拆分、schema 设计、API 设计、RAG 路线、reviewer 闭环、兼容迁移策略这些具体工程决策。

5. 如果项目还没有完全落地，就主动把‘已实现 / 正在实现 / 计划实现’分层说清楚，反而更可信。

四、问法风格参考（已改写整理）

下面这些近期帖子反复出现的问法风格，主要集中在：项目拷打、LangGraph 与 LangChain 对比、RAG 工作流、幻觉治理、MCP、Multi-Agent 协作、SSE/状态管理、Function Calling、Memory 等。本文的问题与回答都已经重新组织改写，不是逐条照搬。

- 牛客《Agent 开发字节面试被问麻了》：问题集中在 agent、mcp、LangGraph、多智能体协作范式、SSE、RAG 流程与优化、工作流。
- 牛客《AI Agent 实习面经 美的》：问题集中在 AI Agent 与 LLM 应用区别、RAG 工作流、幻觉治理、LangChain/LangGraph 对比、Function Calling、记忆机制、ReAct。
- 牛客《大模型算法 Agent 八股整理（秋招面经）》：问题集中在 MCP、Agent vs Workflow、Multi-Agent 协作、记忆空间管理、ReAct、模型能力与框架设计权衡。
- 牛客《AMD 超威半导体 Agent 工具开发岗面经》：问题集中在项目讲解、LangChain/LangGraph、Agent 工作流、RAG、MCP。
- 牛客《百度 AI Agent 开发二面》：强调在复杂状态、失败重试、条件跳转、多轮状态推进场景下，LangGraph 比纯链式编排更适合。